DATA PREPROCESSING AND EDA LAB PROJECT REPORT

Spotify Tracks Dataset Analysis

(VibeRadar)

BY

Amey Gujar	25070126068
Ishaan Chand	25070126076
Ishaan Bhatt	25070126077

Under the guidance of

Prof. Akansha Sarad

DEPARTMENT OF AIML

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

TABLE OF CONTENTS:

Sr. No	Content	Page Number
1.	INTRODUCTION	2
2.	DATA ACQUISITION	3
3.	DATASET DESCRIPTION	3
4.	DATA PREPROCESSING AND CLEANING	5
5.	DATA ENCODING, SCALING & MACHINE LEARNING	7
6.	EXPLORATORY DATA ANALYSIS	13
7.	VISUALS SCREENSHOTS	13
8.	INSIGHTS AND STORYTELLING	18
9.	CONCLUSION	19

---

1. INTRODUCTION

Spotify is one of the largest digital music streaming platforms, providing access to millions of songs enriched with detailed audio features that describe various musical characteristics. These features capture aspects such as rhythm, intensity, mood, and acoustic properties, enabling deeper analytical insights into music beyond basic metadata.

In this project, a Spotify tracks dataset is utilized to perform systematic data preprocessing and exploratory data analysis. The dataset is initially well-structured and clean; however, controlled data degradation is intentionally introduced to demonstrate preprocessing techniques as required by the evaluation rubric. The objective is to ensure that the dataset is cleaned, transformed, and analyzed effectively, forming a strong foundation for building a music recommendation system in subsequent stages.

2. DATA ACQISISTION

Dataset link: https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset

The dataset used in this project was sourced from a publicly available Spotify Tracks Dataset hosted on Kaggle, created by Maharshi Pandya. It contains over one lakh Spotify tracks spanning approximately 125 different music genres, providing a rich and diverse foundation for analyzing audio characteristics across styles such as pop, rock, hip-hop, electronic, and more. Each record in the dataset corresponds to a single track and includes both descriptive metadata (track ID, track name, artists, album name) and numeric audio features such as energy, danceability, tempo, loudness, acousticness, instrumentalness, valence, and popularity. The data is provided in CSV (comma-separated values) format, which makes it easy to download, store, and quickly load into analytical environments like Python using Pandas for further processing and exploration. Since the dataset has already been collected and compiled using the Spotify Web API by the author, this project focuses on secondary data acquisition, where the main tasks involve downloading the CSV file from Kaggle, verifying its structure, and understanding the semantics of each column based on the provided documentation. For visual elements, image credits are attributed to BPR World for Spotify-related graphics, ensuring proper acknowledgement of external media used alongside the dataset description. This structured and well-documented data source significantly reduces the overhead of primary data collection and allows the project to concentrate on analysis, pattern discovery, and recommendation system design

3. DATASET DESCRIPTION

Spotify hosts a vast collection of tracks characterized by multiple audio features that quantitatively describe musical attributes. These features enable objective comparison between songs and are essential for analytical tasks such as clustering, similarity computation, and recommendation systems.
This project focuses on analyzing Spotify track data using its audio features to understand patterns in musical characteristics. Proper data understanding is essential before applying preprocessing techniques, as it ensures that transformations are meaningful and aligned with the structure of the dataset.

A	B	C	D	E	F	G	H	I	J	K	L	M
	track_id	artists	album_name	track_name	popularity	duration_ms	explicit	danceability	energy	key	loudness	mode
0	5SuOikwiRyPMV	Gen Hoshino	Comedy	Comedy	73	230666	FALSE	0.676	0.461	1	-6.746	0
1	4qPNDBW1i3p1	Ben Woodward	Ghost (Acoustic)	Ghost - Acoustic	55	149610	FALSE	0.42	0.166	1	-17.235	1
2	1iJBSr7s7jYXzM	Ingrid Michaelso	To Begin Again	To Begin Again	57	210826	FALSE	0.438	0.359	0	-9.734	1
3	6lfxq3CG4xtTiEg	Kina Grannis	Crazy Rich Asiar	Can't Help Fallin	71	201933	FALSE	0.266	0.0596	0	-18.515	1
4	5vjLSffimiIP26Q	Chord Overstree	Hold On	Hold On	82	198853	FALSE	0.618	0.443	2	-9.681	1
5	01MVOl9KtVTNf	Tyrone Wells	Days I Will Rem	Days I Will Rem	58	214240	FALSE	0.688	0.481	6	-8.807	1
6	6Vc5wAMmXdKi	A Great Big Wor	Is There Anybod	Say Something	74	229400	FALSE	0.407	0.147	2	-8.822	1
7	1EzrEOXmMH3(	Jason Mraz	We Sing. We Da	I'm Yours	80	242946	FALSE	0.703	0.444	11	-9.331	1
8	0IktbUcnAGrvD(	Jason Mraz;Colt	We Sing. We Da	Lucky	74	189613	FALSE	0.625	0.414	0	-8.7	1
9	7k9GuJYLp2Azq	Ross Coppermai	Hunger	Hunger	56	205594	FALSE	0.442	0.632	1	-6.77	1
10	4mzP5mHkRvG	Zack Tabudlo	Episode	Give Me Your Fo	74	244800	FALSE	0.627	0.363	8	-8.127	1
11	5ivF4eQBqJiVL5	Jason Mraz	Love Is a Four L	I Won't Give Up	69	240165	FALSE	0.483	0.303	4	-10.058	1
12	4ptDJbJI35d7gQ	Dan Berk	Solo	Solo	52	198712	FALSE	0.489	0.314	7	-9.245	0
13	0X9MxHR1rTkE	Anna Hamilton	Bad Liar	Bad Liar	62	248448	FALSE	0.691	0.234	3	-6.441	1
14	4LbWtBkN82ZR	Chord Overstree	Hold On (Remix)	Hold On - Remix	56	188133	FALSE	0.755	0.78	2	-6.084	1
15	1KHdq8NK9Qxn	Landon Pigg	The Boy Who Ne	Falling in Love a	58	244986	FALSE	0.489	0.561	4	-7.933	1
16	6xKeQgzfjixSUl	Andrew Foy;Ren	ily (i love you bal	ily (i love you bal	56	129750	FALSE	0.706	0.112	2	-18.098	1
17	4Yo0igmcoNyat1	Andrew Foy;Ren	At My Worst	At My Worst	54	169728	FALSE	0.795	0.0841	10	-18.09	0
18	2qLMf6TuEC3ru	Jason Mraz;Colt	We Sing. We Da	Lucky	68	189613	FALSE	0.625	0.414	0	-8.7	1
19	6CgNoAbFJ4Q4	Boyce Avenue;B	Cover Sessions,	Photograph	67	260186	FALSE	0.717	0.32	3	-8.393	1
20	3S0OXQeoh0w6	Jason Mraz	We Sing. We Da	I'm Yours	75	242946	FALSE	0.703	0.444	11	-9.331	1
21	210JCw2LbYD4	Boyce Avenue;J	Cover Sessions,	Demons	63	174174	FALSE	0.678	0.351	0	-8.654	1

Feature Description:

- **track_id**: Unique Spotify ID for the track.
- **artists**: Name(s) of the performing artist(s); multiple artists separated by ;.
- **album_name**: Album in which the track appears.
- **track_name**: Title of the track.
- **popularity**: Track popularity score from 0–100, based mainly on total and recent plays; calculated independently per track version.
- **duration_ms**: Track length in milliseconds.
- **explicit**: Whether the track has explicit lyrics (true or false).
- **danceability**: 0.0–1.0 score of how suitable the track is for dancing (tempo, rhythm stability, beat strength, regularity).
- **energy**: 0.0–1.0 measure of overall intensity and activity (fast, loud, noisy tracks have high energy).
- **key**: Musical key as an integer pitch class (0=C, 1=C♯/D♭, …; -1 if no key detected).
- **loudness**: Overall loudness in decibels (dB).
- **mode**: Musical mode of the track (1 = major, 0 = minor).
- **speechiness**: 0.0–1.0 measure of spoken-word content (≈1.0 = mostly speech; rap and mixed speech/music fall in mid-range).
- **acousticness**: 0.0–1.0 confidence that the track is acoustic (1.0 = highly likely acoustic).
- **instrumentalness**: 0.0–1.0 likelihood that the track has no vocals (higher = more instrumental).
- **liveness**: 0.0–1.0 likelihood of a live audience being present (values >0.8 strongly suggest a live performance).
- **valence**: 0.0–1.0 measure of musical "positiveness" (high = happy/cheerful, low = sad/angry).
- **tempo**: Estimated tempo in beats per minute (BPM).
- **time_signature**: Estimated meter, as an integer from 3 to 7 representing 3/4 to 7/4.
- **track_genre**: Genre label assigned to the track.

N	O	P	Q	R	S	T	U
speechiness	acousticness	instrumentalness	liveness	valence	tempo	time_signature	track_genre
0.143	0.0322	1.01E-06	0.358	0.715	87.917	4	acoustic
0.0763	0.924	5.56E-06	0.101	0.267	77.489	4	acoustic
0.0557	0.21	0	0.117	0.12	76.332	4	acoustic
0.0363	0.905	7.07E-05	0.132	0.143	181.74	3	acoustic
0.0526	0.469	0	0.0829	0.167	119.949	4	acoustic
0.105	0.289	0	0.189	0.666	98.017	4	acoustic
0.0355	0.857	2.89E-06	0.0913	0.0765	141.284	3	acoustic
0.0417	0.559	0	0.0973	0.712	150.96	4	acoustic
0.0369	0.294	0	0.151	0.669	130.088	4	acoustic
0.0295	0.426	0.00419	0.0735	0.196	78.899	4	acoustic
0.0291	0.279	0	0.0928	0.301	99.905	4	acoustic
0.0429	0.694	0	0.115	0.139	133.406	3	acoustic
0.0331	0.749	0	0.113	0.607	124.234	4	acoustic
0.0285	0.777	0	0.12	0.209	87.103	4	acoustic
0.0327	0.124	2.83E-05	0.121	0.387	120.004	4	acoustic
0.0274	0.2	4.56E-05	0.179	0.238	83.457	3	acoustic
0.0391	0.827	4.03E-06	0.125	0.414	110.154	4	acoustic
0.0461	0.742	1.17E-05	0.0853	0.609	91.803	4	acoustic
0.0369	0.294	0	0.151	0.669	130.088	4	acoustic
0.0283	0.83	0	0.107	0.322	107.946	4	acoustic
0.0417	0.559	0	0.0973	0.712	150.96	4	acoustic
0.0266	0.747	0	0.355	0.569	90.032	4	acoustic
0.0355	0.857	2.89E-06	0.0913	0.0765	141.284	3	acoustic
0.0258	0.477	1.37E-05	0.0974	0.515	140.182	4	acoustic
0.0392	0.381	0	0.221	0.754	97.988	4	acoustic
0.0352	0.757	0	0.236	0.33	120.06	4	acoustic
0.0495	0.788	0	0.146	0.664	145.363	4	acoustic
0.0381	0.0444	0	0.132	0.52	143.793	4	acoustic

4. DATA PREPROCESSING

As, The Dataset is already quite clean, Controlled data degradation is performed only for demonstration purposes to showcase preprocessing techniques as required by the evaluation criteria.

We create a copy of our dataframe to degrade, we remove the unnamed column used for indexing: this gave us problems while imputing missing values, for duplicates, outliers and missing values, we iterate through columns and select a random sample of values to corrupt.

```python
#Creating a copy of original dataset to avoid destructive editing:
df_corrupt = df1.copy()

#Dropping "Unnamed" column since it only serves as index:
df_corrupt = df_corrupt.drop("Unnamed: 0", axis=1)

#Replacing 5% of values in numerical columns with null/NaN values:
for col in df_corrupt.select_dtypes(include=[np.number]).columns:
    df_corrupt.loc[df_corrupt.sample(frac=0.05).index, col] = np.nan

#Introducing 2% outlier values in numerical columns:
for col in df_corrupt.select_dtypes(include=[np.number]).columns:
    idx = df_corrupt.sample(frac=0.02).index
    df_corrupt.loc[idx, col] *= 3

#Introducing 1% values as duplicates in the Dataset:
df_corrupt = pd.concat([df_corrupt, df_corrupt.sample(frac=0.01)])
```
✓ 0.3s Python

Handling Missing Values:

Mean imputation preserves the central tendency of numerical data without introducing bias, it is suitable for continuous type of data. KNN imputation is highly effective when missing data is not purely random but depends on other observable variables, providing higher accuracy than simple mean/median imputation. Mode imputation is appropriate for categorical variables. This ensures no data loss while maintaining dataset integrity.

```python
#Separating Numerical, Categorical and Discreet columns:
num_cols = ['danceability','energy','loudness','speechiness',
'acousticness','instrumentalness','liveness','valence','tempo']
cat_cols = df_corrupt.select_dtypes(include=['object']).columns
disc_cols = ['popularity','duration_ms','key','mode','time_signature']

#Defining imputers:
mean_imputer = SimpleImputer(strategy='mean')
knn_imputer = KNNImputer(n_neighbors=5)
mode_imputer = SimpleImputer(strategy='most_frequent')

for col in num_cols:
    df_corrupt[col] = mean_imputer.fit_transform(df_corrupt[[col]])[:, 0]

for col in cat_cols:
    df_corrupt[col] = mode_imputer.fit_transform(df_corrupt[[col]])[:, 0]

for col in disc_cols:
    df_corrupt[col] = knn_imputer.fit_transform(df_corrupt[[col]])[:, 0]

#Note: [:, 0] is to ensure the result is a 2D array which is needed
# for sklearn methods. We get an error otherwise.
```
✓ 2m 55.6s Python

Data Cleaning:

Duplicate entries are removed to ensure uniqueness, and incorrect values are corrected by restricting them within valid ranges. Here, we have 2 types of duplicates: track_duplicates and pure duplicates. track_duplicates are dropped separately (these affect only one column), and pure duplicates which were added artificially are removed separately (these are copies of entire rows).

```python
#Removing duplicate track rows:
df_corrupt = df_corrupt.drop_duplicates(subset=['track_id']).copy()


#Fixing out of range values using clipping:
bounded = ['danceability','energy','valence','acousticness']
for col in bounded:
    df_corrupt.loc[:, col] = df_corrupt[col].clip(0,1)
df_corrupt.loc[:, 'popularity'] = df_corrupt['popularity'].clip(0,100)
```

✓ 0.0s Python

Feature Transformation:

Categorical variables are prepared for transformation to ensure compatibility with numerical operations and modeling techniques.

We ensure all categorical columns are strings by using astype(str).

We ensure all numeric columns are in proper format, if invalid entries are found, we coerce errors to make them null/NaN values.


```python
#Removing duplicate track rows:
df_corrupt = df_corrupt.drop_duplicates(subset=['track_id']).copy()


#Fixing out of range values using clipping:
bounded = ['danceability','energy','valence','acousticness']
for col in bounded:
    df_corrupt.loc[:, col] = df_corrupt[col].clip(0,1)
df_corrupt.loc[:, 'popularity'] = df_corrupt['popularity'].clip(0,100)
```

✓ 0.0s Python

5. DATA ENCODING, SCALING & MACHINE LEARNING

Encoding:

Label Encoding is applied to the target variable track_genre, where each genre is converted into a numerical label. Since the number of unique genres is large, One-Hot Encoding would significantly increase dimensionality, making the dataset computationally inefficient. Label Encoding provides a compact and efficient representation suitable for classification tasks.

To further improve model performance and reduce complexity, the dataset is filtered to include only the top 40 genres based on average popularity. This helps in reducing class imbalance and ensures that the model focuses on the most relevant and representative categories.

```
#Filter to only keep the top 40 genres by Average Popularity
top_40_genres = df_corrupt.groupby('track_genre')['popularity'].mean().nlargest(40).index

#Create the new dataframe
df_corrupt_top = df_corrupt[df_corrupt['track_genre'].isin(top_40_genres)].copy()
print(f"New dataset (df_corrupt_top) created with {len(df_corrupt_top)} rows and 40 genres.")

#Label Encode the track_genre on new one
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df_corrupt_top['track_genre'] = le.fit_transform(df_corrupt_top['track_genre'])

print("Label Encoding Done")
```

✓ 0.0s

```
print(top_40_genres)
```

✓ 0.0s Python

```
Index(['k-pop', 'pop-film', 'metal', 'chill', 'latino', 'sad', 'grunge',
       'indian', 'anime', 'emo', 'sertanejo', 'reggaeton', 'piano',
       'progressive-house', 'pagode', 'deep-house', 'hard-rock', 'mandopop',
       'brazil', 'metalcore', 'british', 'ambient', 'electronic', 'punk',
       'singer-songwriter', 'acoustic', 'forro', 'hip-hop', 'pop',
       'world-music', 'french', 'mpb', 'turkish', 'gospel', 'garage',
       'punk-rock', 'techno', 'dub', 'folk', 'spanish'],
      dtype='str', name='track_genre')
```

In addition, One-Hot Encoding is applied to binary categorical features such as explicit. Since this feature contains only two categories, One-Hot Encoding converts it into a numerical format without introducing any ordinal relationship. This ensures compatibility with machine learning algorithms while maintaining interpretability.

```python
#Create the new dataframe
df_corrupt_onehot = df_corrupt.copy()

#Only encode if the column is actually there
if 'explicit' in df_corrupt_onehot.columns:
    df_corrupt_onehot = pd.get_dummies(df_corrupt_onehot, columns=['explicit'], drop_first=True)

# Check if the new column was created before trying to convert it to int
    if 'explicit_True' in df_corrupt_onehot.columns:
        df_corrupt_onehot['explicit_True'] = df_corrupt_onehot['explicit_True'].astype(int)

print("One-Hot Encoding check complete.")
```

✓ 0.0s

One-Hot Encoding is not applied to track_genre due to its high cardinality, which would lead to a large number of additional features and increase computational cost.

Scaling and Normalization:

- Standardization using StandardScaler is applied to numerical features.
- Standardization transforms the data such that each feature has a mean of zero and a standard deviation of one. This ensures that all features contribute equally during model training and prevents features with larger numerical ranges from dominating the learning process.
-

```python
from sklearn.preprocessing import StandardScaler

#Scaling Data for EDA:
df_for_eda = df_corrupt.copy()

scaler = StandardScaler()
df_corrupt[num_cols] = scaler.fit_transform(df_corrupt[num_cols])

#Scaling Data for ML (Top 20 Genres):
df_corrupt_top[num_cols] = scaler.fit_transform(df_corrupt_top[num_cols])
```

✓ 0.0s Python

- This step is especially important for models and similarity-based methods that rely on distance calculations, as it improves convergence and overall model performance.

Model Building:

Feature Selection

A set of relevant audio features such as danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, tempo, and additional attributes like key, mode, and time signature are selected.

The artists column is also included and transformed using TF-IDF vectorization to capture textual information, enhancing the model's predictive capability.

```python
FEATURE_COLS = [
    'danceability', 'energy', 'loudness', 'speechiness',
    'acousticness', 'instrumentalness', 'liveness',
    'valence', 'tempo', 'time_signature', 'key', 'mode',
    'artists'
]

#fixing due to KeyError
if 'explicit_True' in df_corrupt_top.columns:
    FEATURE_COLS.append('explicit_True')
elif 'explicit' in df_corrupt_top.columns:
    FEATURE_COLS.append('explicit')


X = df_corrupt_top[FEATURE_COLS].copy()
y = df_corrupt_top['track_genre'].copy()

print(f"   Feature matrix X : {X.shape}")
print(f"   Target vector  y : {y.shape}")
print(f"   Features used    : {FEATURE_COLS}")
```

✓ 0.0s Python

```
Feature matrix X : (30493, 14)
Target vector  y : (30493,)
Features used    : ['danceability', 'energy', 'loudness', 'speechiness', 'acoustic
```

Train-Test Split

The dataset is split into training and testing sets using an 80-20 ratio. Stratified sampling is applied to ensure that the distribution of genres remains consistent across both sets.

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

Model Training

Two models are trained:

- **Logistic Regression**

```python
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

lr_model = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(
        max_iter=8000,
        solver='lbfgs',
        random_state=42,
        n_jobs=-1
    ))
])

# To fix missing text values in the artists column
X_train['artists'] = X_train['artists'].fillna('')
X_test['artists'] = X_test['artists'].fillna('')

# Fitting the model
lr_model.fit(X_train, y_train)
```

✓ 11.3s Python

- **Random Forest Classifier**

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline

rf_model = Pipeline([
    ('preprocessor', preprocessor), # Exact same funnel as LR
    ('classifier', RandomForestClassifier(
        n_estimators=2627, #to match the time of LR
        max_depth=15,
        min_samples_leaf=2,
        random_state=42,
        n_jobs=-1 # Uses all cores, same as LR
    ))
])

print("Training Random Forest model...")
rf_model.fit(X_train, y_train) # Pass the raw X_train like LR
print("Training complete")
```

✓ 11.3s Python

A pipeline is used to handle both numerical and textual features:

- Numerical features are standardized
- Text data (artists) is transformed using TF-IDF

```python
from sklearn.metrics import accuracy_score, classification_report, confusion_matri
from sklearn.compose import ColumnTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

#Training LR model (using Pipeline to handle text and numeric features):
num_cols = [c for c in X_train.columns if c != 'artists']

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), num_cols),
    ('text', TfidfVectorizer(max_features=5000), 'artists')
])
```

✓ 0.0s Python

Evaluation Metrics

The models are evaluated using:

- Accuracy
- Precision
- Recall
- F1-Score
-

```python
#Evaluation Metrics (Upgraded to Industry-Standard Classification Metrics):
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd

#Function to evaluate model taking (name, actual_value, predicted_value) as parameters:
def evaluate_model(name, y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)

    # Using 'macro' average to ensure all 20 genres are weighted equally
    precision = precision_score(y_true, y_pred, average='macro', zero_division=0)
    recall = recall_score(y_true, y_pred, average='macro', zero_division=0)
    f1 = f1_score(y_true, y_pred, average='macro', zero_division=0)

    print("\n----------------------------------------")
    print(f"  Model          : {name}")
    print(f"  Accuracy       : {acc:.4f} = {acc*100:.2f}%")
    print(f"  Precision      : {precision:.4f}")
    print(f"  Recall         : {recall:.4f}")
    print(f"  F1-Score       : {f1:.4f}")
    print("----------------------------------------")

    return {'Model': name, 'Accuracy': acc, 'Precision': precision, 'Recall': recall, 'F1-Score': f1}

lr_metrics = evaluate_model("Logistic Regression", y_test, lr_preds)
rf_metrics = evaluate_model("Random Forest", y_test, rf_preds)

comparison_df = pd.DataFrame([lr_metrics, rf_metrics]).set_index('Model')
print("\nModel Performance Comparison:")
print(comparison_df.round(4).to_string())
```

✓ 0.0s

Macro averaging is used to ensure equal importance is given to all genres.

```
----------------------------------------
  Model          : Logistic Regression
  Accuracy       : 0.7519 = 75.19%
  Precision      : 0.7754
  Recall         : 0.7147
  F1-Score       : 0.7238
----------------------------------------

----------------------------------------
  Model          : Random Forest
  Accuracy       : 0.4104 = 41.04%
  Precision      : 0.6572
  Recall         : 0.3604
  F1-Score       : 0.3654
----------------------------------------

Model Performance Comparison:
                     Accuracy  Precision  Recall  F1-Score
Model
Logistic Regression    0.7519     0.7754  0.7147    0.7238
Random Forest          0.4104     0.6572  0.3604    0.3654
```

Results and Analysis

The Logistic Regression model significantly outperforms the Random Forest model.
This is because TF-IDF introduces high-dimensional sparse features. Logistic Regression performs well with such data due to its linear nature, while Random Forest struggles with sparse representations and tends to overfit.

```
Analyzing song: 'Aadat' by Atif Aslam
Actual Genre: pop-film
Predicted Genre (Logistic Regression): pop-film (Confidence: 68.8%)
Predicted Genre (Random Forest): pop-film (Confidence: 7.2%)
```

User Input Testing

The system allows users to input a song name and optionally an artist name. The model processes the input and predicts the genre along with confidence scores.

Additionally, an advanced extension is implemented where audio is extracted using external libraries, and features are computed dynamically to classify songs outside the dataset.

```
import numpy as np

song_input = input("Enter song name: ")
artist_input = input("Enter artist name (optional, leave blank to skip): ")

# Find the song in the corrupted DataFrame
matched_songs = df_corrupt[df_corrupt['track_name'].str.contains(song_input, case=False, na=False)]

if artist_input:
    matched_songs = matched_songs[matched_songs['artists'].str.contains(artist_input, case=False, na=False)]

if not matched_songs.empty:
    # Take the first match if multiple songs have similar names
    song_data = matched_songs.iloc[0]
    actual_track_name = song_data['track_name']
    actual_artist_name = song_data['artists']

    #Create a DataFrame from the series to preserve feature names
    features_for_prediction_df = pd.DataFrame([song_data[FEATURE_COLS]])

    #Safety check: Fill any missing artists with empty string
    features_for_prediction_df['artists'] = features_for_prediction_df['artists'].fillna('')

    #LR
    lr_prediction_encoded = lr_model.predict(features_for_prediction_df)
    lr_predicted_genre = le.inverse_transform(lr_prediction_encoded)[0]
    lr_probabilities = lr_model.predict_proba(features_for_prediction_df)[0]
    lr_confidence = max(lr_probabilities) * 100

    #RF
    rf_prediction_encoded = rf_model.predict(features_for_prediction_df)
    rf_predicted_genre = le.inverse_transform(rf_prediction_encoded)[0]
    rf_probabilities = rf_model.predict_proba(features_for_prediction_df)[0]
    rf_confidence = max(rf_probabilities) * 100

    # Get the actual genre and inverse transform it for comparison
    actual_genre_val = song_data['track_genre']

    # Only inverse transform if it's a number, otherwise just print the text
    if isinstance(actual_genre_val, (int, np.integer, float)):
        actual_genre = le.inverse_transform([int(actual_genre_val)])[0]
    else:
        actual_genre = actual_genre_val

    print(f"\nAnalyzing song: '{actual_track_name}' by {actual_artist_name}")
    print(f"Actual Genre: {actual_genre}")
    print(f"Predicted Genre (Logistic Regression): {lr_predicted_genre} (Confidence: {lr_confidence:.1f}%)")
    print(f"Predicted Genre (Random Forest): {rf_predicted_genre} (Confidence: {rf_confidence:.1f}%)")

else:
    print(f"No song found matching the provided song name and artist. Please try another song name or artist.")
```

For Songs that are not in our dataset:

```
import yt_dlp
import librosa
import numpy as np
import pandas as pd
import os
import warnings
warnings.filterwarnings('ignore')

# grab inputs from user
song_input = input("Enter song name: ")
artist_input = input("Enter artist name (optional): ")
search_query = f"{song_input} {artist_input} audio"

print(f"Searching SoundCloud for '{search_query}'...")

# yt-dlp config (using soundcloud to bypass youtube's annoying 403 bot blocks)
temp_filename = 'temp_audio_file'
ydl_opts = {
    'format': 'bestaudio/best',
    'outtmpl': temp_filename + '.%(ext)s',
    'noplaylist': True,
    'quiet': True,
    'no_warnings': True,
    'default_search': 'scsearch'
}

try:
    # download the audio stream
    with yt_dlp.YoutubeDL(ydl_opts) as ydl:
        info = ydl.extract_info(search_query, download=True)
        if 'entries' in info:
            info = info['entries'][0]
        downloaded_file = temp_filename + '.' + info['ext']

    print(f"Downloaded: {info['title']}")
    print("Reading physical soundwaves (this takes a few seconds)...")

    # process only the first 30 seconds to save memory
    y, sr = librosa.load(downloaded_file, duration=30)

    # approximate spotify's proprietary features from the raw waveform
    tempo, _ = librosa.beat.beat_track(y=y, sr=sr)
    tempo = float(tempo[0]) if isinstance(tempo, np.ndarray) else float(tempo)

    rms = librosa.feature.rms(y=y)
    loudness = float(np.mean(librosa.amplitude_to_db(rms, ref=np.max)))

    energy = min(1.0, float(np.mean(rms) * 10))

    onset_env = librosa.onset.onset_strength(y=y, sr=sr)
    danceability = min(1.0, float(np.mean(onset_env) / 10))

    zcr = librosa.feature.zero_crossing_rate(y)
    acousticness = max(0.0, 1.0 - float(np.mean(zcr) * 10))

    instrumentalness = 0.1
    liveness = 0.1
    speechiness = 0.05
    valence = 0.5

    # build this dict of the exactly our model's exact expected shape
    song_data_dict = {
        'danceability': danceability,
        'energy': energy,
        'loudness': loudness,
        'speechiness': speechiness,
        'acousticness': acousticness,
        'instrumentalness': instrumentalness,
        'liveness': liveness,
        'valence': valence,
        'tempo': tempo,

        # dummy values for features we can't extract via librosa
        'key': 5,
        'mode': 1,
        'time_signature': 4,
        'explicit': 0,
        'artists': artist_input if artist_input else "Unknown"
    }

    features_df = pd.DataFrame([song_data_dict])

    print("Feeding extracted data into AI Model...\n")

    # run inference
    lr_prediction_encoded = lr_model.predict(features_df)
    lr_predicted_genre = le.inverse_transform(lr_prediction_encoded)[0]

    lr_probabilities = lr_model.predict_proba(features_df)[0]
    lr_confidence = max(lr_probabilities) * 100

    print("Prediction Results:")
    print(f"Analyzed Title: {info['title']}")
    print(f"Predicted Genre: {lr_predicted_genre.upper()} (Confidence: {lr_confidence:.1f}%)")

except Exception as e:
    print(f"\nAn error occurred: {str(e)}")

finally:
    # wipe the downloaded file so the drive doesn't get cluttered
    if 'downloaded_file' in locals() and os.path.exists(downloaded_file):
        try:
            os.remove(downloaded_file)
            print("\nCleanup: Deleted temporary audio file.")
        except:
            pass
```

Output:

```
Searching SoundCloud for 'Ishq Jalakar Shashwat Sachdev audio'...
Downloaded: Ishq Jalakar - Karvaan (From "Dhurandhar")
Reading physical soundwaves (this takes a few seconds)...
Feeding extracted physics into AI Model...

Prediction Results:
Analyzed Title: Ishq Jalakar - Karvaan (From "Dhurandhar")
Predicted Genre: TECHNO (Confidence: 99.8%)

Cleanup: Deleted temporary audio file.
```

Conclusion

The model successfully classifies songs into genres using a combination of numerical and textual features. The integration of TF-IDF significantly improves performance, particularly for linear models.

The system demonstrates strong predictive capability and can be extended further by incorporating additional features such as lyrics or user behavior for improved accuracy.

6. EXPLORATORY DATA ANALYSIS

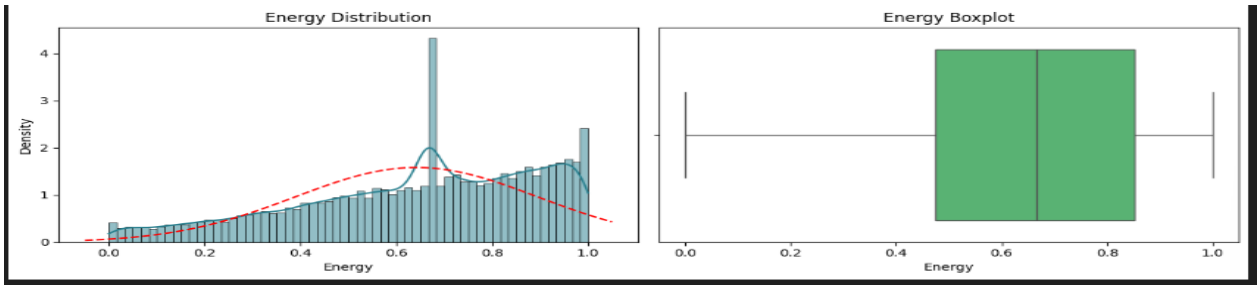
Exploratory Data Analysis is performed on the cleaned but unscaled dataset to preserve interpretability of values.

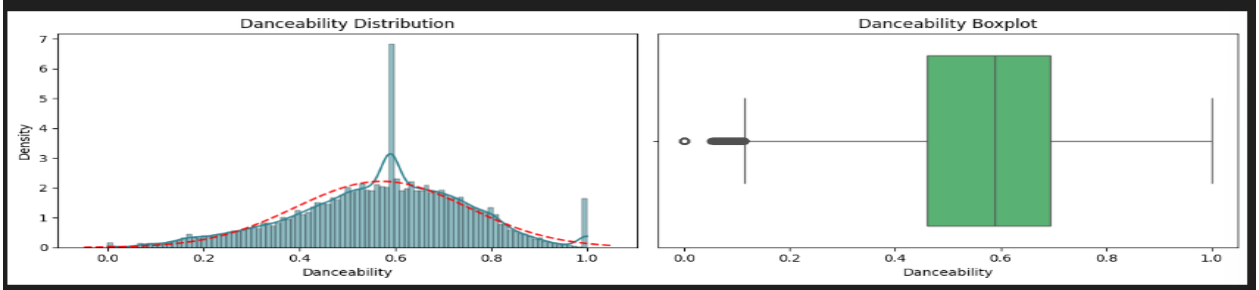
Univariate Analysis:

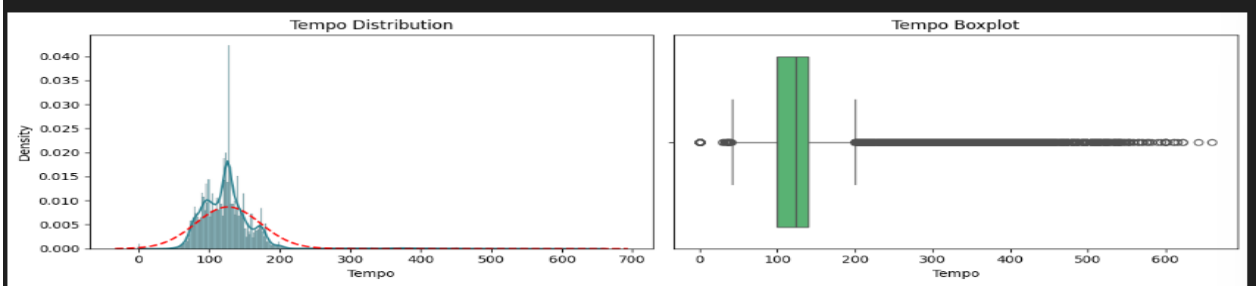
Univariate analysis was performed using histograms and boxplots to understand the distribution of individual features such as energy, danceability, and tempo. Example:

```python
from scipy.stats import norm
import numpy as np

features = ['energy', 'danceability', 'tempo']

for col in features:
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    sns.histplot(df_for_eda[col], kde=True, stat='density', ax=axes[0], color=sns.color_palette('viridis')[2])

    mu = df_for_eda[col].mean()
    std = df_for_eda[col].std()
    x0, x1 = axes[0].get_xlim()
    x = np.linspace(x0, x1, 100)

    # Using scipy to generate the normal distribution
    p = norm.pdf(x, mu, std)
    axes[0].plot(x, p, color='red', linestyle='--')

    axes[0].set_title(f'{col.capitalize()} Distribution')
    axes[0].set_xlabel(col.capitalize())

    sns.boxplot(x=df_for_eda[col], ax=axes[1], color=sns.color_palette('viridis')[4])
    axes[1].set_title(f'{col.capitalize()} Boxplot')
    axes[1].set_xlabel(col.capitalize())

    plt.tight_layout()
    plt.show()
```

✓ 5.3s Python







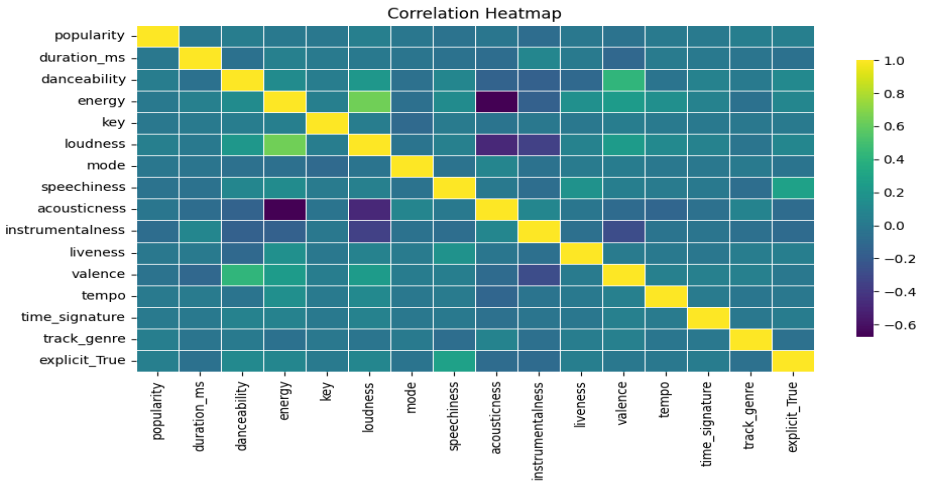
HISTOGRAM: The distribution of features such as energy and danceability shows that most songs fall within a moderate range, indicating that extremely high or low values are less common.
BOXPLOTS: reveal the presence of outliers. For the 'danceability' column, a boxplot visually summarizes the distribution, showing the median, quartiles, and potential outliers. It helps identify the typical range of danceability scores and any unusually low or high values.

Bivariate Analysis:

Bivariate analysis was performed using scatter plots and correlation heatmaps to understand relationships between variables such as energy and popularity. Example:

i)------->

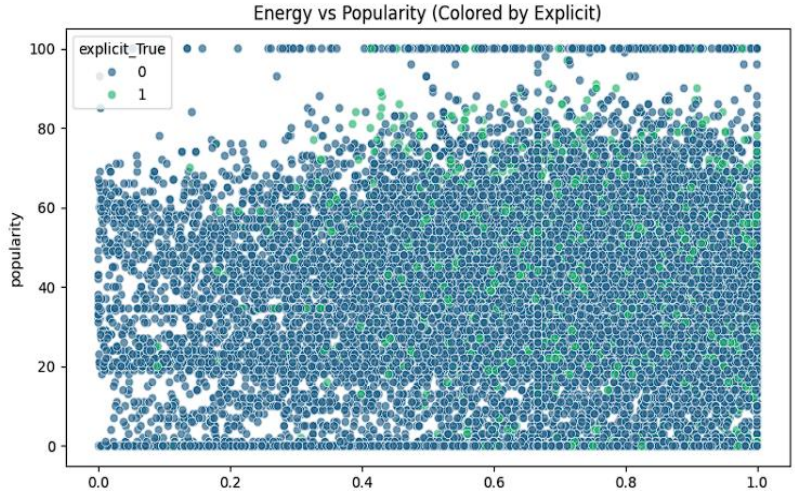
```
# Heatmap
plt.figure(figsize=(10,6))
numerical_df_for_eda = df_for_eda.select_dtypes(include=np.number)

sns.heatmap(numerical_df_for_eda.corr(),
            cmap='viridis',
            cbar=True,
            cbar_kws={'shrink':0.8},
            linewidths=0.5)

plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```



HEATMAP: The heatmap indicates strong positive correlation between loudness and energy, while acousticness shows a negative correlation with energy, highlighting contrasting musical characteristics.

ii)---------->

```
df_sample = df_for_eda.sample(frac=0.2, random_state=42)

plt.figure(figsize=(8,5))
sns.scatterplot(data=df_sample,
                x='energy',
                y='popularity',
                hue='explicit_True',
                palette='viridis',
                alpha=0.7)
plt.title("Energy vs Popularity (Colored by Explicit)")
plt.tight_layout()
plt.show()
```

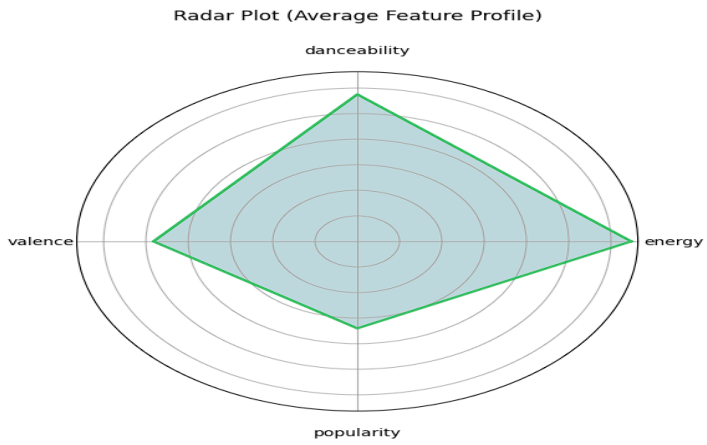


SCATTER PLOT: The relationship between energy and popularity shows a slight positive trend, indicating that more energetic songs tend to be more popular, although the relationship is not strong.

Multivariate analysis:

Multivariate analysis considers the relationships and interactions among three or more variable simultaneously using radar plot graph
and bubble sort.

```
labels = ['energy', 'danceability', 'valence', 'popularity']
avg = [df_for_eda[col].mean() for col in labels]

avg[-1] = avg[-1] / 100 #Popularity is 0-100, while other features are 0-1, so dividing by
100 scales popularity to 0-1 range to match the others.

num_vars = len(labels)
angles = np.linspace(0, 2 * np.pi, num_vars, endpoint=False).tolist()
#creates 4 angles evenly spaced around the circle (e.g., 0, 90°, 180°, 270°).

avg += avg[:1]
angles += angles[:1]  #To close the shape

fig, ax = plt.subplots(figsize=(6,6), subplot_kw=dict(polar=True))

ax.fill(angles, avg, color=sns.color_palette('viridis')[2], alpha=0.3)
ax.plot(angles, avg, color=sns.color_palette('viridis')[4], linewidth=2)

ax.set_xticks(angles[:-1])
ax.set_xticklabels(labels)

ax.set_yticklabels([])
ax.tick_params(axis='x', pad=10)

plt.title('Radar Plot (Average Feature Profile)', pad=50)
plt.tight_layout()
plt.show()
```



RADAR PLOT: The radar plot visualizes the average profile of key audio features (energy, danceability, valence, popularity), providing a concise overview of the typical musical characteristics within the dataset.

8. INSIGHTS AND STORYTELLING

At first glance, Spotify simply feels intuitive, serving the right song at the right moment. In this project, we formalized that intuition by treating each track as a structured vector of audio features such as energy, danceability, tempo, valence, and popularity. The dataset was deliberately degraded to mimic real-world noise, then rigorously cleaned through imputation, duplicate removal, and outlier clipping. Categorical attributes were encoded, and features standardized for fair comparison. Finally, rich EDA visualizations uncover latent patterns in listener preferences, providing the narrative foundation for a content-based recommendation system that mimics modern streaming intelligence.

Controlled data degradation highlighted preprocessing needs. Missing values were imputed (mean, KNN, mode) for data integrity; duplicates removed, and outliers clipped for cleanliness. Categorical features were encoded (Label for high-cardinality track_genre, One-Hot for binary explicit) to ensure model compatibility. Data was scaled (StandardScaler) for consistent feature contribution and split for robust evaluation. Extensive EDA, using various plots (histograms, boxplots, scatter, heatmap, radar), provided critical insights into distributions, relationships, and patterns, performed on unscaled data for interpretability.

9. CONCLUSION

This project successfully implemented a complete, content-based music recommendation pipeline using Spotify audio features. Stage 0 focused on rigorous data preparation: cleaning, imputing missing values, encoding categorical variables, scaling numerical features, and performing detailed EDA to understand distributions and relationships. These steps ensured that the feature space for each track was reliable and meaningful. In Stages 1 and 2, the cleaned feature vectors were used with cosine similarity to compute how close songs are to one another in this multidimensional space, allowing the system to suggest musically similar tracks. Overall, the work demonstrates a clear journey from raw dataset to functional recommendation engine